

Topological Signal Processing and Learning

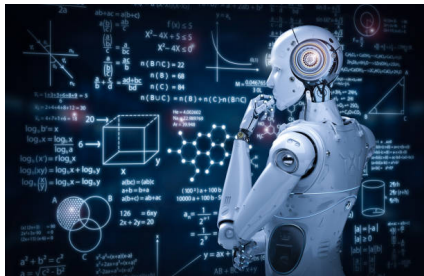
Course Introduction and Graphs

Elvin Isufi

Intelligent Systems Department

May 15, 2026

Machine learning vs. human learning



► Machine learning

- ⇒ need for **huge amounts** of data
- ⇒ typically purely **data-driven** (inductive)
- ⇒ **not always good** generalization capabilities

► Human learning

- ⇒ **no need** for huge amounts of data
- ⇒ fit knowledge into an **existing structured representation** (deductive)
- ⇒ **excellent** generalization capabilities

Machine learning vs. human learning

How can we teach machines to learn as humans?

Machine learning vs. human learning

How can we **teach machines** to **learn** as humans?

Data Is The Fossil Fuel Of AI, It Will Get Exhausted: Ilya Sutskever

© DECEMBER 14, 2024 ▲ OFFICECHAI TEAM

Machine learning vs. human learning

How can we teach machines to learn as humans?

Data Is The Fossil Fuel Of AI, It Will Get Exhausted: Ilya Sutskever

© DECEMBER 14, 2024 ▲ OFFICECHAI TEAM

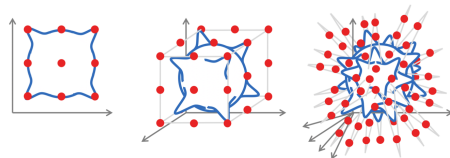


fig.credit: Bronstein et. al

Machine learning vs. human learning

How can we teach machines to learn as humans?

Data Is The Fossil Fuel Of AI, It Will Get Exhausted: Ilya Sutskever

© DECEMBER 14, 2024 ▲ OFFICECHAI TEAM

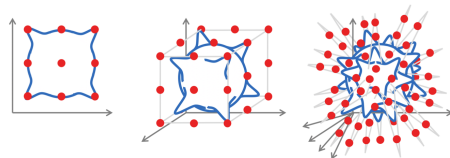


fig.credit: Bronstein et. al

- Use knowledge
 - ⇒ **inductive bias**: incorporate models about solution space or data-generation process
 - ⇒ **examples**: Bayesian models, regularization, **relational data structure**

Machine learning vs. human learning

How can we **teach machines** to **learn** as humans?

Data Is The Fossil Fuel Of AI, It Will Get Exhausted: Ilya Sutskever

© DECEMBER 14, 2024 ▲ OFFICECHAI TEAM

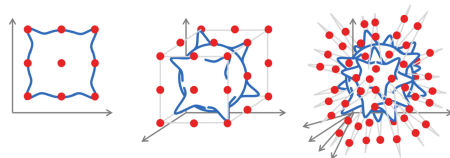


fig.credit: Bronstein et. al

- Use knowledge
 - ⇒ **inductive bias**: incorporate models about solution space or data-generation process
 - ⇒ **examples**: Bayesian models, regularization, **relational data structure**
- In this course: **relational inductive bias** using **topological data structure**

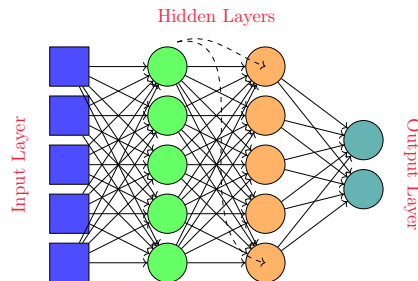
Curse of dimensionality

- ▶ Learning with **multi-layer perceptrons**

- ▶ **Criticalities**

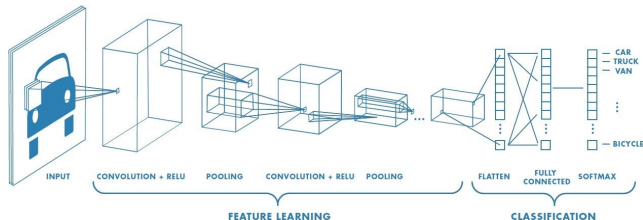
- ⇒ Curse of dimensionality
- ⇒ Generalization capabilities
- ⇒ Explainability
- ⇒ Adversarial attacks

- ▶ Propagation rule $\mathbf{x}_\ell = \sigma(\mathbf{W}_\ell \mathbf{x}_{\ell-1} + \mathbf{b}_\ell)$
 - ⇒ if layers have dimensions $\sim \mathcal{O}(N)$ $\Rightarrow \dim(\mathbf{W}_l) \sim \mathcal{O}(N^2)$, e.g., $N = 1000 \rightarrow \mathcal{O}(10^6)$
 - ⇒ complexity $\mathcal{O}(N^2)$
- ▶ The data **structure** helps overcome the curse of dimensionality



Presence of relational structure in data

- **Successful example:** convolutional neural networks (CNNs)



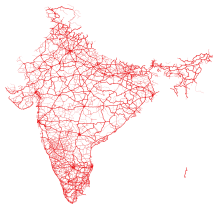
- Data contain **relational structures** that can be leveraged for processing and learning
- **Embed** invariance / equivariances of **signal domain** into the **learning** process
 $\text{Grid domain} \iff \text{convolution operator is translation equivariant}$
- Exploit **local** relations

Beyond Euclidean relations in data structure

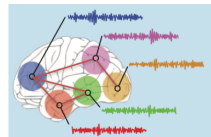
- ▶ **General problem:** Most of the data do **not** reside over a **grid** or **Euclidean** space
- ▶ **Data with irregular structure**
 - ⇒ Networked data
 - ⇒ complex systems
 - ⇒ hidden dependencies (correlation, causation)



Social network



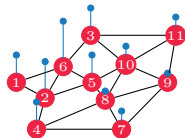
Road network



Brain recordings

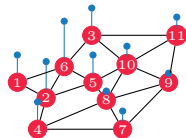
Pairwise irregular relations in data

- **Graphs** are a **universal** language to capture **pairwise** relationships
 - ⇒ **Nodes** are entities (users, cities)
 - ⇒ **Edges** are connections (friendships, roads)
 - ⇒ **Signals** are values on nodes (opinions, traffic level)



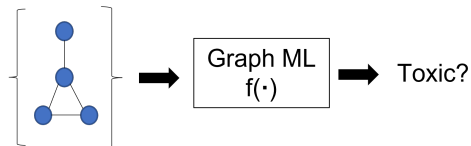
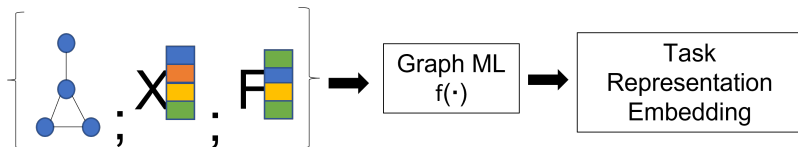
Pairwise irregular relations in data

- **Graphs** are a **universal** language to capture **pairwise** relationships
 - ⇒ **Nodes** are entities (users, cities)
 - ⇒ **Edges** are connections (friendships, roads)
 - ⇒ **Signals** are values on nodes (opinions, traffic level)
- **Graph-induced deep learning**: $\Phi(\mathcal{G}, \mathbf{X}, \mathcal{H}) \rightarrow \mathbf{Z}$
 - ⇒ graph is our **relational inductive** bias
 - ⇒ $\Phi(\cdot)$ is **local**



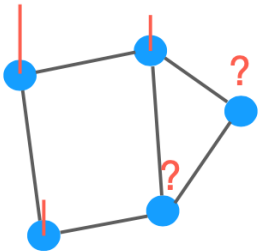
What is graph (topological) machine learning?

- **Goal:** Find the best function for the task we want to solve; i.e., find a mapping function $f(\cdot)$



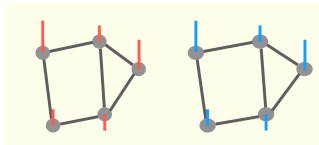
Landmark tasks with graph machine learning

Signal-level tasks



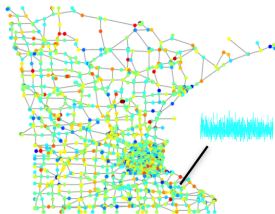
Signal reconstruction/interpolation

Ex. Find missing values in a sensor network



Signal classification

Ex. Health/non-healthy patient from EEG recordings

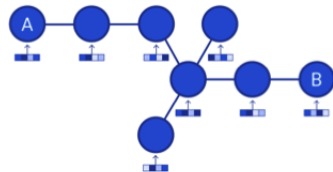
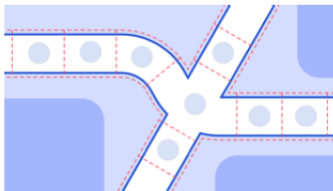
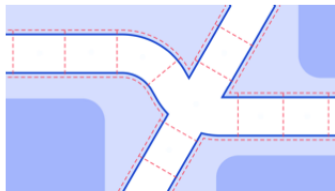


Signal prediction

Ex. Predict traffic

Landmark tasks with graph machine learning

Examples of Signal-level tasks

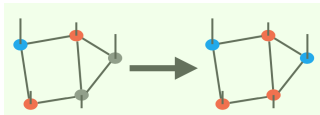


- Predict estimation time-of-arrival on GoogleMaps
 - ⇒ **Nodes:** road segments
 - ⇒ **Edges:** proximity between segments
 - ⇒ **Task:** forecast signal

Image credit Deepmind: <https://www.deepmind.com/publications/eta-prediction-with-graph-neural-networks-in-google-maps>

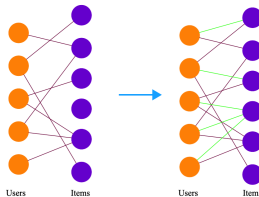
Landmark tasks with graph machine learning

Structure-level tasks



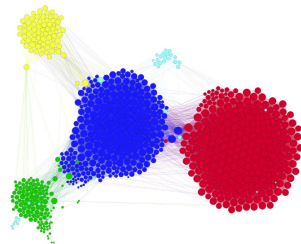
Node classification

Ex. Classify users of a social network from their friends labels



Link prediction

Ex. Recommend items

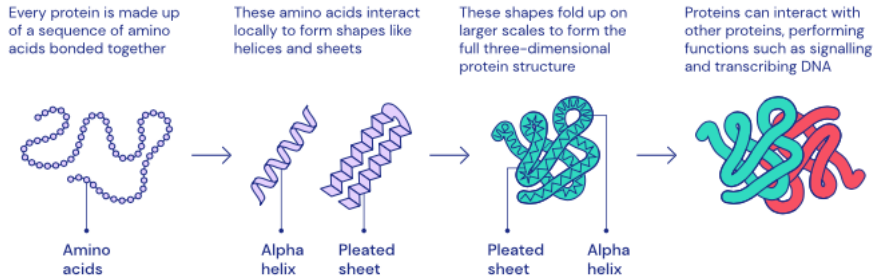


Clustering & community detection

Ex. Finding communities in facebook (same interest)

Landmark tasks with graph machine learning

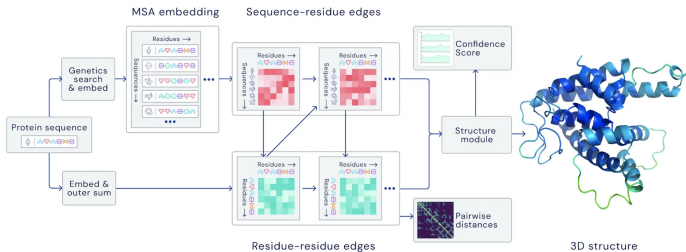
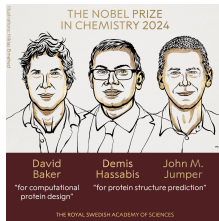
Examples of Node-level tasks



- Predict 3D protein structure based on its amino-acid sequence
 - ⇒ **Nodes:** amino-acids in a protein sequence
 - ⇒ **Edges:** proximity between amino-acids (residues)
 - ⇒ **Task:** predict node positions

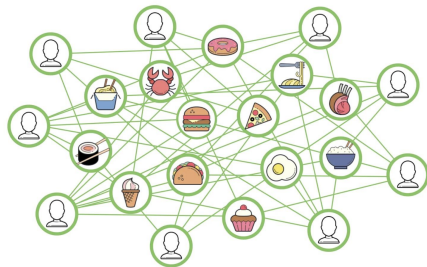
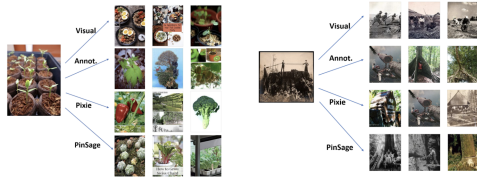
Image credit: Deepmind: <https://www.deepmind.com/blog/alphafold-using-ai-for-scientific-discovery-2020>

Breakthrough in 2025



Landmark tasks with graph machine learning

Examples of Edge-level tasks



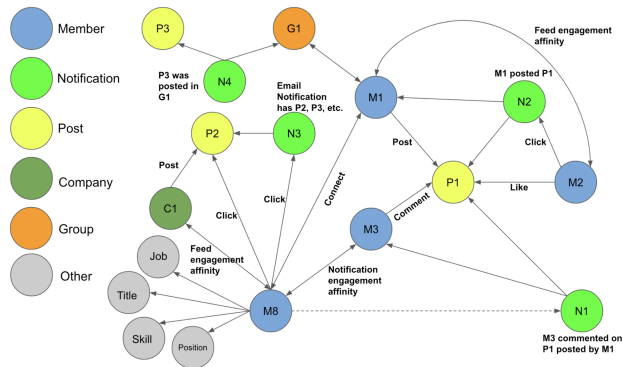
- Pin recommendation at **Pinterest** & food recommendation at **UberEats**
⇒ Billions of nodes, users, pins, restaurants, dishes - all in real time

⇒ <https://medium.com/pinterest-engineering/pinsage-a-new-graph-convolutional-neural-network-for-web-scale-recommender-systems-88795a107f48>

⇒ <https://www.uber.com/en-NL/blog/uber-eats-graph-learning/>

Breakthrough in 2025

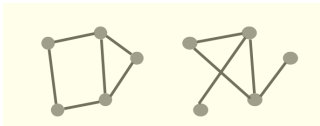
Graph ML powers LinkedIn



► <https://arxiv.org/abs/2402.11139>

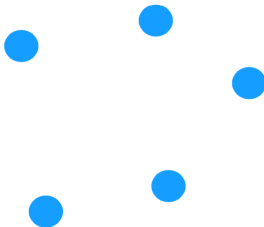
Landmark tasks with graph machine learning

Graph-level tasks



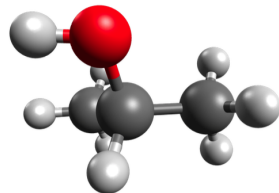
Graph classification

Ex. Given a collection of structures,
find which water network is resilient or
not



Graph identification

Ex. Given datapoints (stock features)
find how they influence each other

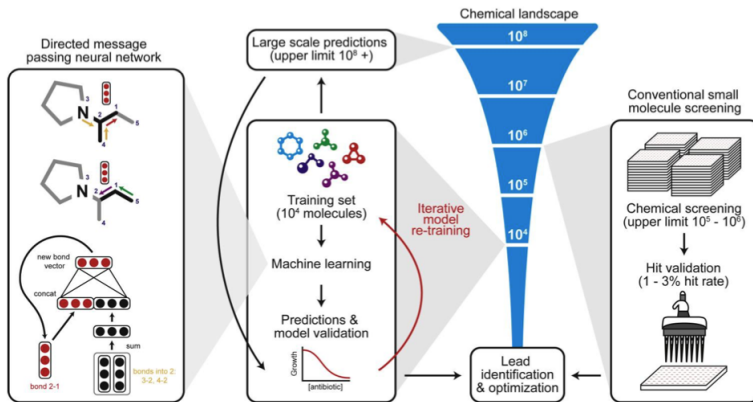


Graph generation

Ex. Given antibacterial activity level,
generate a new molecule

Landmark tasks with graph machine learning

Example graph-level tasks



- Analyse and process molecular structure to generate new antibiotics

⇒ https://www.youtube.com/watch?v=d7CkzgHi_tE&list=PLcZ0nmyqlala5P0skgoEkX07U29yFdYHF&index=5

Image credit: A deep learning approach to antibiotic discovery

Topologies Beyond Graphs

Beyond *pairwise* data relationships

- Graphs are a **limited** example of topological space

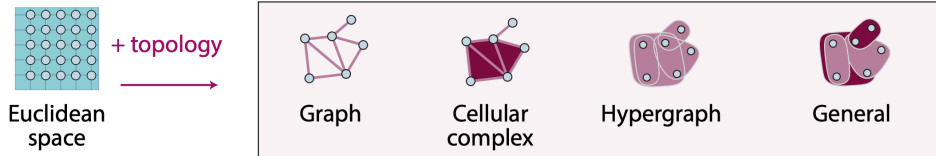


fig.credit: Sanborn. al

Beyond *pairwise* data relationships

- ▶ Graphs are a **limited** example of topological space

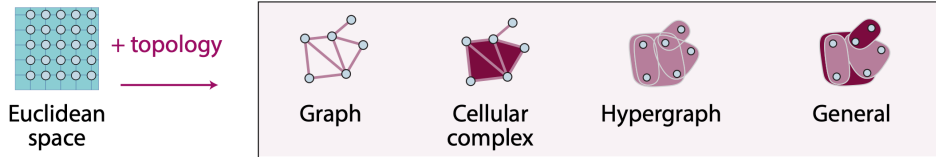
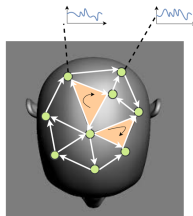


fig.ccredit: Sanborn. al

- ▶ **Graphs** encode only **pairwise** relations
 - ⇒ Several brain regions activate together
 - ⇒ Multiple co-collaborators in a single project



Complex systems



Coauthorship network

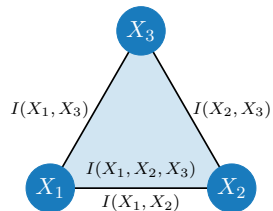
Beyond *pairwise* data relationships

- Motivation for **encoding multiway** relations

- **Information-theoretic perspective**

⇒ **Pairwise mutual information:**

$$I(X_1, X_2) = H(X_1) + H(X_2) - H(X_1, X_2)$$



Beyond *pairwise* data relationships

- Motivation for **encoding multiway** relations

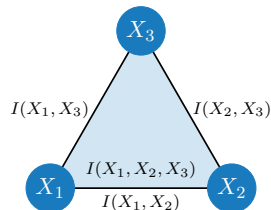
- **Information-theoretic perspective**

⇒ **Pairwise mutual information:**

$$I(X_1, X_2) = H(X_1) + H(X_2) - H(X_1, X_2)$$

- **Triple-wise mutual information:**

$$I(X_1, X_2, X_3) = H(X_1) + H(X_2) + H(X_3) - H(X_1, X_2) - H(X_2, X_3) - H(X_1, X_3) + \underbrace{H(X_1, X_2, X_3)}_{\text{tot. entropy of 3 var.}}$$

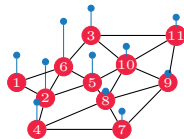


Mutual information between 3 random variables **cannot be reduced to pairwise** mutual information

- We need **higher-order** or **multiway** structures

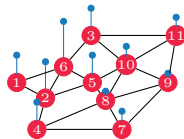
Beyond *node-centric* signal processing

- Graph signal processing, graph neural networks and alternatives are **principled** approaches to **process signals** defined on the **nodes**



Beyond *node-centric* signal processing

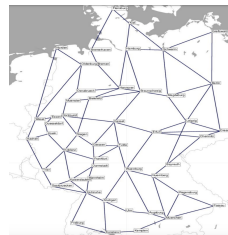
- ▶ Graph signal processing, graph neural networks and alternatives are **principled** approaches to **process signals** defined on the **nodes**
- ▶ Increasing demand to process **signals on tuples** such as **flows**
- ▶ **Joint** processing of **node + edge** signals in critical infrastructure networks



Water networks



Power networks



Communication networks

Signal over topological spaces

Definition. A **signal** defined **over** a **topological space** $\mathcal{S}$ is a **mapping** from the elements of the topological space to the real domain

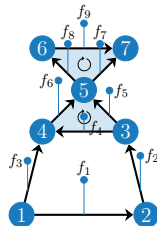
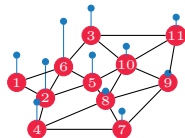
$$f : \mathcal{S} \rightarrow \mathbb{R}$$

Signal over topological spaces

Definition. A **signal** defined **over** a **topological space** $\mathcal{S}$ is a **mapping** from the elements of the topological space to the real domain

$$f : \mathcal{S} \rightarrow \mathbb{R}$$

- ▶ **Ex.** graph signal defined over the nodes
- ▶ **Ex.** edge signals defined over the edges (pairs of nodes)



What is this course about?

Understand, analyze and evaluate **principled** and **tractable** deep learning techniques for processing topological signals

What is this course about?

Understand, analyze and evaluate **principled** and **tractable** deep learning techniques for processing topological signals

- ▶ **LO.1:** **Recognize** key challenges in processing and learning from complex topological data
- ▶ **LO.2:** **Compare** and **explain** the inner working mechanisms of different methods tailored to processing topological data
- ▶ **LO.3:** **Analyse** different topological deep learning techniques
- ▶ **LO.4:** **Implement** guided solutions to practical applications

Learning activity: When do we need topologies beyond graphs?

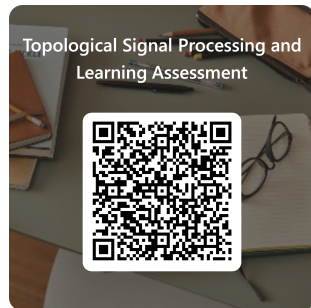
Q1.1: A research team wants to model a collaboration dataset where three or more authors jointly write papers, and the goal is to learn from these group interactions directly.

► Instructions

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

► Logistics

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Answer the question and justify your choice
- ⇒ **Product:** One submitted response per student + justification to your peers



<https://forms.office.com/e/yTag1Njapt>

Course logistics: Lectures

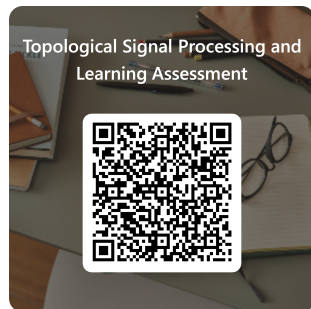
Day / Time	Day 1 - Tu - 26/05	Day 2 - We - 27/05	Day 3 - Th - 28/05	Day / Time (Friday only)	Day 5 - Fr - 29/05
09:00-10:15	Course Into	Topological Structures	Topological VAR Models	09:00-10:30	Spectral Analysis
10:15-10:30	Break	Break	Break	10:30-10:45	Break
10:30-11:45	Graph Convents	Topological Convolution	Topological State-Spaces	10:45-11:45	Guest Lecture
11:45-12:45	Lunch	Lunch	Lunch	11:45-12:45	Lunch
12:45-14:30	GNNs	Topological Deep Learning	Topological-xLSTM Models	12:45-13:00	Closing Remarks
14:30-14:45	Break	Break	Break		
14:45-16:00	Lab 1: GNNs	Lab 2: TNNs	Lab 3: Topological Time Series		

- Mind the slight **shift** on **Friday**
- **Contact** Dr. Elvin Isufi: e.isufi-1@tudelft.nl

Course logistics: assessment

For those interested in a certificate

- ▶ **Deadline: 29/05: 17:00**
 - ⇒ Fill in **any** time
 - ⇒ You need 60% of points to pass
- ▶ Every lecture / lab will have 2 sanity check questions
 - ⇒ We will dedicate time during the lectures in the form of a LA
 - ⇒ Feel free to discuss with colleagues
 - ⇒ **Goal:** You **understand** the concept



<https://forms.office.com/e/yTag1Njapt>

Graph terminology

Knowledge Representation *via* Pairwise Relationships

► Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$

⇒ A set $\mathcal{V}$ of vertices or nodes $\mathcal{V} = \{1, 2, 3, 4, 5, 6, 7\}$ in the figure

⇒ Connected by a set $\mathcal{E} = \{(i, j)\}$ of edges or links

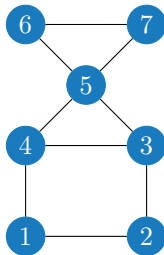
► Representations

⇒ adjacency matrix $\mathbf{A}$

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

⇒ incidence matrix $\mathbf{B}_1$

$$\mathbf{B}_1 = \left[\begin{array}{cccccccccc} \overbrace{1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0}^{\text{edges}} \\ \underbrace{-1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0}_{\text{nodes}} \\ 0 \quad 0 \quad -1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \\ 0 \quad -1 \quad 0 \quad -1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \\ 0 \quad 0 \quad 0 \quad 0 \quad -1 \quad -1 \quad 1 \quad 1 \quad 0 \\ 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad -1 \quad 0 \quad 1 \\ 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad -1 \quad -1 \end{array} \right]$$



Graph signals and graph shift operator

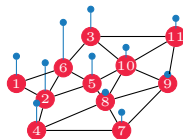
Definition (GSO). A graph shift operator (GSO) for a graph $\mathcal{G}$ is a **matrix** $\mathbf{S} \in \mathbb{R}^{N \times N}$ satisfying

$$[\mathbf{S}]_{ji} = s_{ji} = 0 \text{ whenever } (i, j) \notin \mathcal{E} \text{ for } i \neq j$$

► **Ex. of GSO:** Adjacency matrix $\mathbf{A}$

⇒ Graph Laplacian matrix $\mathbf{L} = \mathbf{D} - \mathbf{A} = \mathbf{B}_1 \mathbf{B}_1^\top$

$$\mathbf{L} = \begin{bmatrix} 2 & -1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & 0 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 & -1 & 0 & 0 & 0 & -1 & -1 \\ 0 & -1 & 0 & 3 & 0 & 0 & -1 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 4 & -1 & 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & -1 & 0 & -1 & 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 3 & -1 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & -1 & 0 & -1 & 5 & -1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 \\ 0 & 0 & -1 & 0 & -1 & 0 & 0 & -1 & -1 & 5 & -1 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 3 \end{bmatrix}$$



Graph signals and graph shift operator

Definition (GSO). A graph shift operator (**GSO**) for a graph $\mathcal{G}$ is a **matrix** $\mathbf{S} \in \mathbb{R}^{N \times N}$ satisfying

$$[\mathbf{S}]_{ji} = s_{ji} = 0 \text{ whenever } (i, j) \notin \mathcal{E} \text{ for } i \neq j$$

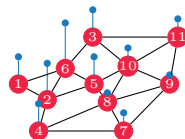
► **Ex. of GSO:** Adjacency matrix $\mathbf{A}$

$$\Rightarrow \text{Graph Laplacian matrix } \mathbf{L} = \mathbf{D} - \mathbf{A} = \mathbf{B}_1 \mathbf{B}_1^\top$$

Definition. A **graph signal** is a **mapping** from the vertex set the real domain $f : \mathcal{V} \rightarrow \mathbb{R}$.

► **Graph signals** $\Rightarrow$ Associate a **value to each node**

$$\Rightarrow \text{collect in vector } \mathbf{x} = [x_1, x_2, \dots, x_N]^\top \in \mathbb{R}^N$$



How to build graphs?

- ▶ We have two main strategies to build graphs
- ▶ **Physical networks**: the edges are already available
 - ⇒ **Road network**: nodes are cross roads, edges are streets connecting them
 - ⇒ **Robot networks**: nodes are agents, edges are communication links
 - ⇒ **Brain networks**: nodes are brain areas, edges are structural connectivities

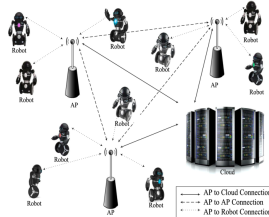
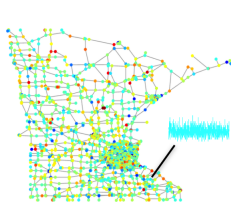
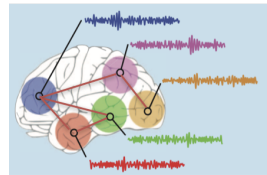


image credit Varma et al



- ▶ **Abstract networks**: build a **proximity structure** from data points
 - ⇒ These can be dependencies, similarities, causal influence etc.

Abstract networks

- Depend on proximities between data points
 - ⇒ Let $\mathbf{x}_1, \dots, \mathbf{x}_N$ be N datapoints each of dimensions F ; i.e., $\mathbf{x}_i \in F$
 - ⇒ **Treat** each datapoint as **a node**
 - ⇒ **Connect** two datapoints i and j based on a **proximity/similarity/distance metric**, e.g.,:

$$sim(i, j) = g(\mathbf{x}_i, \mathbf{x}_j)$$

- ⇒ Leads to a **fully-connected** graph
- Various choices imply various structures
 - ⇒ **Similarity**: correlation; dependency; causation; covariance;
 - ⇒ **Sparsity**: threshold; k - nearest neighbors; ϵ -ball neighborhood
 - ⇒ **Hyperparameter**: k, ϵ

ϵ - neighbourhood networks

- Let $\text{dist}(\mathbf{x}_i, \mathbf{x}_j)$ represent a distance between datapoints (e.g., physical or Euclidean distance)
 - ⇒ Set edge weights as the **Kernelized** version of the distance; e.g., Gaussian kernel

$$A_{ij} = \begin{cases} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2}\right) = \exp\left(-\frac{\text{dist}^2(\mathbf{x}_i, \mathbf{x}_j)}{2\sigma^2}\right) & \text{if } \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \leq \epsilon \\ 0 & \text{otherwise} \end{cases}$$

- ⇒ Build edge set as $\mathcal{E} = \{(i, j) : \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \leq \epsilon\}$
- ⇒ σ is a hyperparameter of the kernel; ϵ defines the radius
- ⇒ $\epsilon \rightarrow 0$ sparser graphs; $\epsilon \rightarrow \infty$ densely connected graphs

Graph ML via Regularization

Signal behavior over graphs in real applications

- ▶ Measurements in sensor networks, e.g., temperature, traffic
- ▶ Neighboring nodes tend to have similar values / labels

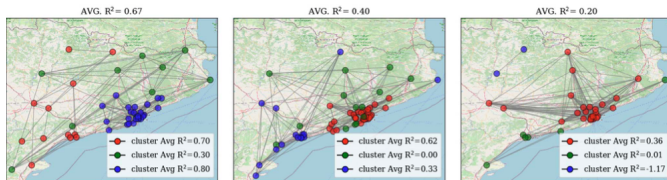
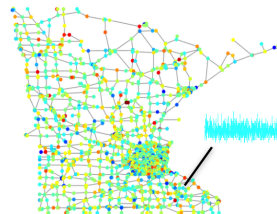


Image credit: Graph Signal Reconstruction Techniques for IoT Air Pollution Monitoring Platforms

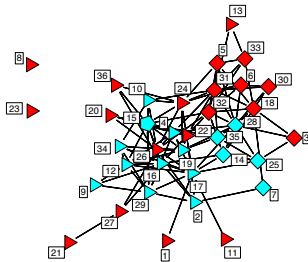


Traffic signal

- ▶ This prior assumption can be used to **remove noise**, **interpolate missing values**, **classify nodes**.

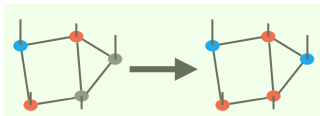
Signal behavior over graphs in real applications

- ▶ Network of working relationships among lawyers
 - ▶ Nodes are $N = 36$ partners, edges indicate partners worked together



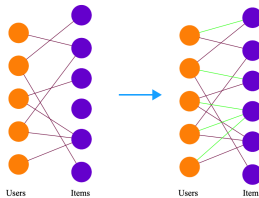
- ⇒ Data includes various node-level attributes $\{x_i\}_{i \in \dots}$ including
 - ⇒ Type of practice, i.e., litigation (red) and corporate (cyan)
- ▶ Suspect lawyers collaborate more with peers in same legal practice
 - ⇒ Knowledge of collaboration useful in predicting type of practice

Applications



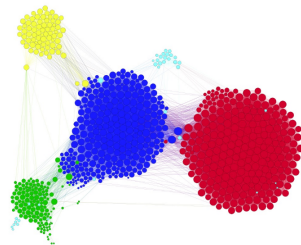
Node classification

Ex. Classify users of a social network from their friends labels



Link prediction

Ex. Recommend items



Clustering & community detection

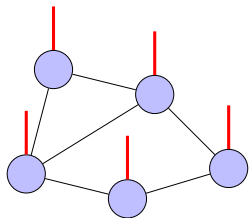
Ex. Finding communities in facebook (same interest)

- **Observation:** Signal has similar values in connected nodes!
⇒ **Question:** How can we leverage it to solve graph ML tasks?

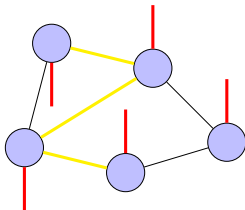
- ▶ **Observation:** Signal has similar values in connected nodes!
⇒ **Question:** How can we leverage it to solve graph ML tasks?
- ▶ We need to **measure** the signal **variability** over the graph!

Signal variation over graphs

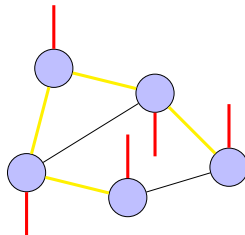
- How fast does the signal change over the graph?
 - ⇒ Identify patterns, characterise filters
 - ⇒ Initial thought: count sign changes



constant, no sign change



slow-varying, 3 sign changes

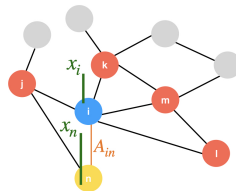
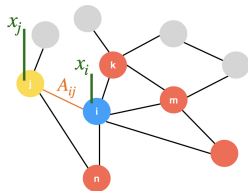


high-varying, 4 sign changes

Signal variation over graphs

- ▶ Gradient at a node
- ▶ **Edge derivative:** variation of signal $\mathbf{x}$ w.r.t an edge $e_{i,j} = (i,j)$ at node i

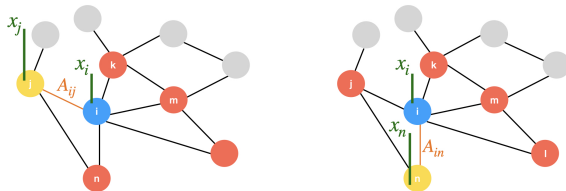
$$\frac{\partial \mathbf{x}}{\partial e_{i,j}}|_i = \sqrt{A_{ij}}(x_i - x_j)$$



Signal variation over graphs

- ▶ Gradient at a node
- ▶ **Edge derivative:** variation of signal $\mathbf{x}$ w.r.t an edge $e_{i,j} = (i, j)$ at node i

$$\frac{\partial \mathbf{x}}{\partial e_{i,j}}|_i = \sqrt{A_{ij}}(x_i - x_j)$$



- ▶ **Node gradient:** vector collecting all edge derivatives for node i

$$\nabla_i \mathbf{x} = \left[\frac{\partial \mathbf{x}}{\partial e_{i,1}}|_i, \dots, \frac{\partial \mathbf{x}}{\partial e_{i,n}}|_i \right]^\top$$

Signal variation over graphs

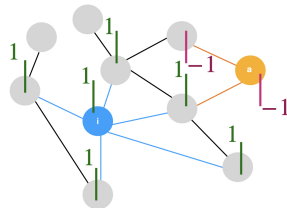
- **Local variability:** norm-2 of the gradient vector

$$\|\nabla_i \mathbf{x}\|_2 = \left(\sum_{j \in i} A_{ij} (x_i - x_j)^2 \right)^{1/2}$$

⇒ Larger $\|\nabla_i \mathbf{x}\|_2$ signal varies more around the node

- **Global variation:** sum all square local variations

$$S_2(\mathbf{x}) = \frac{1}{2} \sum_{i \in \mathcal{V}} \|\nabla_i \mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{L} \mathbf{x}$$



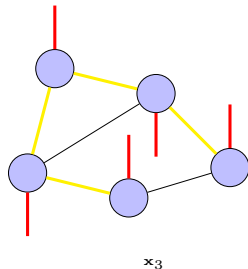
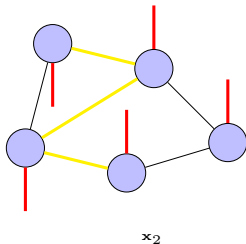
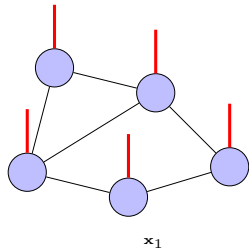
$$S_2(\mathbf{x}) = \frac{1}{2} \sum_{i \in \mathcal{V}} \|\nabla_i \mathbf{x}\|_2^2 = \frac{1}{2} \sum_{i \in \mathcal{V}} \sum_{j \in i} A_{ij} (x_i - x_j)^2 = \sum_{(i,j) \in \mathcal{E}} A_{ij} (x_i - x_j)^2$$

Signal variation over graphs

- Signal smoothness measure

$$S_2(\mathbf{x}) = \frac{1}{2} \sum_{i \in E} \|\nabla_i \mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

quantifies how much the signal changes over the graph



- $S_2(\mathbf{x}_1) = 0 < S_2(\mathbf{x}_2) < S_2(\mathbf{x}_3)$

Tikhonov Regularization

- ▶ A graph signal $\mathbf{x}$ is **smooth** over a graph: $S_2(\mathbf{x}) = \mathbf{x}^\top \mathbf{L} \mathbf{x}$ is low.
- ▶ We observe a noisy version of the signal $\mathbf{y} = \mathbf{x} + \mathbf{n}$. The goal is to recover $\mathbf{x}$ from the observation $\mathbf{y}$.
- ▶ Because noise is **random** it has a **high variability**

Tikhonov Regularization

- ▶ A graph signal $\mathbf{x}$ is **smooth** over a graph: $S_2(\mathbf{x}) = \mathbf{x}^\top \mathbf{L} \mathbf{x}$ is low.
- ▶ We observe a noisy version of the signal $\mathbf{y} = \mathbf{x} + \mathbf{n}$. The goal is to recover $\mathbf{x}$ from the observation $\mathbf{y}$.
- ▶ Because noise is **random** it has a **high variability**
- ▶ We can solve the **Tikhonov regularization problem**

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{y} - \mathbf{x}\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

Tikhonov Regularization

- ▶ A graph signal $\mathbf{x}$ is **smooth** over a graph: $S_2(\mathbf{x}) = \mathbf{x}^\top \mathbf{L} \mathbf{x}$ is low.
- ▶ We observe a noisy version of the signal $\mathbf{y} = \mathbf{x} + \mathbf{n}$. The goal is to recover $\mathbf{x}$ from the observation $\mathbf{y}$.
- ▶ Because noise is **random** it has a **high variability**
- ▶ We can solve the **Tikhonov regularization problem**

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{y} - \mathbf{x}\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

$\Rightarrow$ **Fitting term** $f(\mathbf{x}, \mathbf{y}) = \|\mathbf{y} - \mathbf{x}\|_2^2$ – signal close to observations

Tikhonov Regularization

- ▶ A graph signal $\mathbf{x}$ is **smooth** over a graph: $S_2(\mathbf{x}) = \mathbf{x}^\top \mathbf{L} \mathbf{x}$ is low.
- ▶ We observe a noisy version of the signal $\mathbf{y} = \mathbf{x} + \mathbf{n}$. The goal is to recover $\mathbf{x}$ from the observation $\mathbf{y}$.
- ▶ Because noise is **random** it has a **high variability**
- ▶ We can solve the **Tikhonov regularization problem**

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{y} - \mathbf{x}\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

$\Rightarrow$ **Fitting term** $f(\mathbf{x}, \mathbf{y}) = \|\mathbf{y} - \mathbf{x}\|_2^2$ – signal close to observations

$\Rightarrow$ **Regularization term** $g(\mathbf{x}, \mathbf{L}) = \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$ – **desired prior**: signal variation is small in adjacent nodes

Tikhonov Regularization

- We can solve the Tikhonov regularization problem

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{y} - \mathbf{x}\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

- Scalar $\gamma > 0$ controls the trade-off
 - $\Rightarrow \gamma \rightarrow 0$ – we only solve fitting term – less desired prior
 - $\Rightarrow \gamma \gg 0$ – we only solve the desired prior – less fitting

Tikhonov Regularization

- More generally we have **Partial and noisy observations at some nodes** $N' < N$: $\mathbf{y} = \mathbf{M}(\mathbf{x} + \mathbf{n})$

⇒ Matrix $\mathbf{M} \in \{0, 1\}^{N' \times N}$ tells which nodes are observed $M_{n',n} = 1$ and which not $M_{n',n} = 0$

Tikhonov regularizer problem

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{M}(\mathbf{y} - \mathbf{x})\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

In the optimization problem, $\mathbf{y}$ is padded with zeros where the signal value is not observed to match the dimensions.

Tikhonov Regularization

- More generally we have **Partial and noisy observations at some nodes** $N' < N$: $\mathbf{y} = \mathbf{M}(\mathbf{x} + \mathbf{n})$

⇒ Matrix $\mathbf{M} \in \{0, 1\}^{N' \times N}$ tells which nodes are observed $M_{n', n} = 1$ and which not $M_{n', n} = 0$

Tikhonov regularizer problem

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{M}(\mathbf{y} - \mathbf{x})\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

In the optimization problem, $\mathbf{y}$ is padded with zeros where the signal value is not observed to match the dimensions.

- **General problem formulation**

⇒ If $\mathbf{x}$ is a **graph signal**: interpolate missing values from partial noisy observations $\mathbf{y}$

⇒ If $\mathbf{x}$ is a **node label**: classify noise from noisy partial labels $\mathbf{y}$

Tikhonov Regularization

- More generally we have **Partial and noisy observations at some nodes** $N' < N$: $\mathbf{y} = \mathbf{M}(\mathbf{x} + \mathbf{n})$

⇒ Matrix $\mathbf{M} \in \{0, 1\}^{N' \times N}$ tells which nodes are observed $M_{n', n} = 1$ and which not $M_{n', n} = 0$

Tikhonov regularizer problem

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{M}(\mathbf{y} - \mathbf{x})\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x}$$

In the optimization problem, $\mathbf{y}$ is padded with zeros where the signal value is not observed to match the dimensions.

- **General problem formulation**

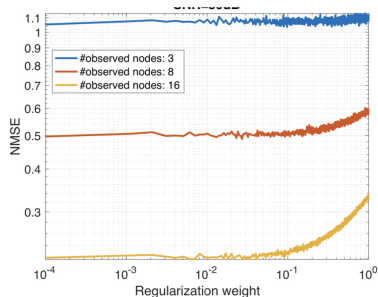
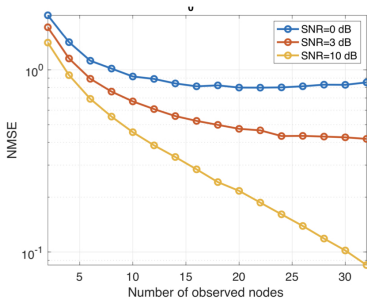
⇒ If $\mathbf{x}$ is a **graph signal**: interpolate missing values from partial noisy observations $\mathbf{y}$

⇒ If $\mathbf{x}$ is a **node label**: classify noise from noisy partial labels $\mathbf{y}$

- **Closed form solution** (convex problem) $\hat{\mathbf{x}} = (\mathbf{M}^\top \mathbf{M} + \gamma \mathbf{L})^{-1} \mathbf{M}^\top \mathbf{M} \mathbf{y}$

Tikhonov regularizer (example)

- Temperature data on $N = 32$ weather stations
 - ⇒ 5-nearest neighbor graph
 - ⇒ Temperature recordings are graph signals
 - ⇒ task: recover signals from a few noisy observations



Learning activity: Lawyers network and regularization trade-off

A network connects 36 lawyers. Nodes are partners, edges indicate they have worked together, and the goal is to classify each lawyer as mainly **litigation** or **corporate**. Some labels are known, others are missing.

- **Q1.2:** In a graph-regularized classification approach, what is the role of the trade-off parameter γ in

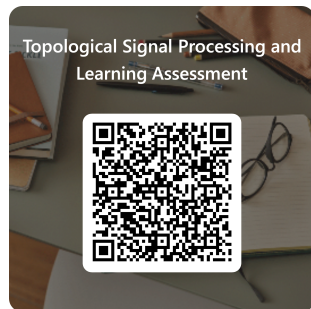
$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{M}(\mathbf{y} - \mathbf{x})\|_2^2 + \gamma \mathbf{x}^\top \mathbf{L} \mathbf{x} ?$$

- **Instructions**

- ⇒ Scan the QR code to open the activity
- ⇒ Work individually first
- ⇒ Then discuss your answer in pairs or groups of 3
- ⇒ Submit your final answer in Microsoft Forms

- **Logistics**

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Answer the question and justify your choice
- ⇒ **Product:** One submitted response per student + justification to your peers



<https://forms.office.com/e/yTag1Njapt>

Further reading

- ▶ (Barabasi) Network science <http://networksciencebook.com>
⇒ Ch1 and Ch2
- ▶ (Isufi et al) Graph filters for processing and learning from network data
⇒ Ch1 (excluding Sec. 1.2)
- ▶ (Ma and Tang) Deep Learning on Graphs https://web.njit.edu/~ym329/dlg_book/
⇒ Ch 2 (excluding Sec. 2.3.3; 2.4; 2.5)
- ▶ Some of the material in these slides (figures, tables etc.) were taken from these courses:
 - ⇒ CS2242: <https://web.stanford.edu/class/cs224w/>
 - ⇒ ICASSP 2022 short-course: <https://iie.fing.edu.uy/personal/flarroca/teaching/signal-processing-and-learning-from-network-data/>
- ▶ Signal regularization
 - ⇒ (Isufi et al) Graph filters for processing and learning from network data [Ch, 1, 2](#)
 - ⇒ (Isufi et al) Graph filters for signal processing and machine learning on graphs [Sec, VI.D](#)

Review: mathematical notation and concepts

► Vector norms

► Function mapping a vector $\mathbf{x} \in \mathbb{R}^N$ to a real number

► Used thoroughly in optimization theory:

► Generic l_p norm for $p \geq 1$: $\|\mathbf{x}\|_p = (\sum_{n=1}^N |\mathbf{x}_n|^p)^{\frac{1}{p}}$

► Commonly used vector norms:

$\Rightarrow l_2$ norm: $\|\mathbf{x}\|_2 = (\sum_{n=1}^N \mathbf{x}_n^2)^{\frac{1}{2}}$: Indicates the energy of the vector

$\Rightarrow l_1$ norm: $\|\mathbf{x}\|_1 = (\sum_{n=1}^N |\mathbf{x}_n|)$: Indicates the sparsity of the vector

$\Rightarrow l_\infty$ norm: $\|\mathbf{x}\|_\infty = \max_{n=1:N} |\mathbf{x}_n|$: Maximum absolute element

► (Isufi et al) Graph filters for processing and learning from network data

$\Rightarrow$ Appendix A

Review: mathematical notation and concepts

► Optimization problem

- Minimizes a function of a parameter, in our case Θ
- Backbone of machine learning problems
- General formulation

$$\begin{aligned} & \min_{\Theta} f_0(\Theta) \\ & \text{subject to } f_l(\Theta) \leq 0, \quad l = 1 : L \\ & \quad \quad \quad f_m(\Theta) \leq 0, \quad m = 1 : M \end{aligned}$$

- Problem above is constrained (Ex: SVM,)
 - We also have unconstrained problems (Ex: Ordinary Least Squares, Logistic Regression)
 - Depending on the nature of $f(\Theta)$, we have different approaches
 - $f(\Theta)$ can be convex (Least Squares), or non-convex (Neural Networks)
 - Plenty of algorithms available for solving such problems
-
- (Isufi et al) Graph filters for processing and learning from network data
→ Appendices B and C

Review: mathematical notation and concepts

► Regularized problems

- Optimization problems incorporating priors over Θ
- Backbone of machine learning problems
- General formulation (unconstrained)

$$\min_{\Theta} f(\mathbf{x}, \mathbf{y}, \Theta) + \lambda g(\Theta), \quad \lambda > 0$$

- $f(\mathbf{x}, \mathbf{y}, \Theta)$ measures goodness of fit, $g(\Theta)$ incorporates the prior (also called regularizer)
- This prevents overfitting by increasing variance of prediction, influenced by λ
- Example of a regularized least square problem

$$\min_{\Theta} ||\mathbf{y} - \Theta^T \mathbf{x}||_2^2 + \lambda ||\Theta||_2^2$$

- l_2 norm prior enforces small norm (low energy) of the desired Θ

- (Isufi et al) Graph filters for processing and learning from network data
⇒ Appendices B and C